Sécurisation du Code en C++ : Immutabilité et Tripartition d'Etcheverry

Thème principal : Initiation au développement (C++) - Bonnes pratiques et structure de code

Synthèse exécutive : Ce cours aborde deux piliers fondamentaux pour écrire du code propre et robuste en C++ : d'une part, l'utilisation du mot-clé 'const' pour garantir l'immutabilité des variables et sécuriser les paramètres de fonctions à la compilation ; d'autre part, la méthodologie de la 'Tripartition d'Etcheverry', une structure algorithmique rigoureuse qui sépare la phase d'entrée (déclaration et saisie), la phase de traitement (logique pure), et la phase de sortie (restitution).

1. La Constance en C++ : Le mot-clé 'const'

En C++, la gestion de la mémoire et la robustesse du code reposent sur la rigueur du typage et de l'accès aux données. Par défaut, une variable classique peut être modifiée n'importe où dans le programme. Cependant, certaines valeurs fondamentales (comme Pi, un taux de TVA, ou des configurations système) ne doivent jamais varier. Le mot-clé 'const' permet de verrouiller une variable dès sa déclaration en écrivant 'const type nomVariable = valeur;'. Si une modification accidentelle est tentée plus loin dans le code, le compilateur bloque la compilation, éliminant ainsi une grande famille de bugs. Appliqué aux paramètres de fonctions, 'const' garantit que la fonction agit en lecture seule.

- Immutabilité : Empêche la modification d'une variable après son initialisation.
- Sécurité à la compilation : Le compilateur rejette toute tentative de réaffectation d'une constante.
- Documentation explicite : Informe clairement les développeurs qu'une valeur est fixe.

Cas concret enseignant : *Imaginez que vous écrivez un programme pour un distributeur de billets. Le solde minimum autorisé est de 0 euro. Si vous ne déclarez pas cette limite avec 'const', un développeur distrait pourrait accidentellement modifier cette variable critique lors d'une mise à jour, permettant aux clients de vider les caisses. Avec 'const', le compilateur joue le rôle de garde-fou physique.*

2. La Tripartition d'Etcheverry : Structurer son algorithme

La 'Tripartition d'Etcheverry' est une convention méthodologique enseignée pour structurer proprement le code et éliminer le code spaghetti. Elle impose un ordre séquentiel strict dans chaque fonction ou programme principal. En Phase 1, on prépare toutes les variables nécessaires et on récupère les entrées. En Phase 2, on réalise les calculs, conditions et boucles de manière isolée. Enfin, en Phase 3, on retourne ou on affiche le résultat. Mélanger ces étapes nuit gravement à la maintenabilité et à la lisibilité du code.

- Tripartition : Découpage obligatoire d'une fonction en trois phases chronologiques.
- Phase 1 (Entrées) : Déclaration, initialisation et saisie des variables.
- Phase 2 (Traitement) : Logique algorithmique pure, sans I/O utilisateur.
- Phase 3 (Sorties) : Restitution ou retour du résultat final.

Cas concret enseignant : *Mélanger les saisies, les calculs et les affichages au milieu d'une fonction, c'est comme essayer de faire cuire des pâtes tout en allant faire les courses et en dressant la table en même temps. La tripartition vous force à préparer vos ingrédients d'abord (Phase 1), à cuisiner ensuite (Phase 2), et à servir enfin (Phase 3).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi utiliser 'const' pour un paramètre de fonction si on sait déjà qu'on ne va pas le modifier dans notre code ?
Réponse du professeur : C'est une question de sécurité et de maintenance sur le long terme. Même si vous savez aujourd'hui que vous ne le modifiez pas, un autre développeur (ou vous-même dans six mois) pourrait introduire une modification par erreur. Le 'const' force le compilateur à vous bloquer.

Question étudiant : Est-ce qu'on a le droit de déclarer une variable locale à l'intérieur d'une boucle en Phase 2 selon la tripartition ?
Réponse du professeur : Oui, les variables de travail strictement nécessaires aux calculs itératifs (comme un compteur de boucle 'int i = 0') se déclarent au moment opportun dans la Phase 2, mais les données globales de la fonction et les entrées doivent être préparées en Phase 1.

■ Exercice & Démonstration d'Application

Énoncé : Écrire une fonction C++ nommée 'calculerPrixTTC' qui prend en paramètre un prix HT (double) et un taux de TVA (double). Ces deux paramètres doivent être protégés contre toute modification. La fonction doit calculer le prix TTC et le retourner, en respectant scrupuleusement la tripartition d'Etcheverry.

Résolution pas à pas :
Étape 1 : Définir la signature de la fonction avec les paramètres constants ('const double prixHT' et 'const double tauxTVA'). Étape 2 : Phase 1 de la tripartition -> Déclarer et initialiser la variable locale 'prixTTC' à 0.0. Étape 3 : Phase 2 de la tripartition -> Effectuer le calcul logique : 'prixTTC = prixHT * (1.0 + tauxTVA);'. Étape 4 : Phase 3 de la tripartition -> Retourner le résultat final avec 'return prixTTC;'.

Résultat : double calculerPrixTTC(const double prixHT, const double tauxTVA) { // PHASE 1 : Déclaration et Initialisation double prixTTC = 0.0; // PHASE 2 : Traitement prixTTC = prixHT * (1.0 + tauxTVA); // PHASE 3 : Restitution return prixTTC; }

■ Auto-Évaluation Flash d'Examen

Q : Que se passe-t-il si on tente de modifier une variable déclarée avec 'const' ?

R : Le compilateur rejette le code et génère une erreur de compilation, empêchant la création de l'exécutable.

Q : Quelles sont les trois phases de la tripartition d'Etcheverry, dans l'ordre ?

R : 1. Phase d'Entrée (Déclaration et Saisie), 2. Phase de Traitement (Logique pure), 3. Phase de Sortie (Restitution et Affichage/Retour).

Q : Pourquoi la Phase 2 (Traitement) ne doit-elle pas contenir d'instructions d'affichage (comme cout) ?

R : Pour garantir la séparation des préoccupations (separation of concerns), rendre la fonction testable unitairement et éviter de coupler la logique métier avec les interfaces utilisateurs.